01 · SYSTEM OVERVIEW

System Architecture

How the user interface, the pipeline controller, the processing modules and the shared utilities fit together.

User
researcher / lab

PRESENTATION LAYER

Web Interface

`app.py · Gradio`

- Upload microscopy image
- Parameter sliders
- Intermediate result viewer
- Results & CSV export

BUILT WITH

Gradio · Matplotlib

image + params →

← results

APPLICATION LAYER

Pipeline Controller

`core/pipeline.py`

- Validates the input
- Runs the stages in order
- Caches U-Net output
- Collects all results

BUILT WITH

Python 3.8+ · dataclasses

CORE PROCESSING MODULES · core/

1. **Preprocessing** — Contrast & noise normalisation — OpenCV
2. **U-Net Segmentation** — Per-pixel cell probability map — PyTorch
3. **Mask Refinement** — Threshold + morphology clean-up — OpenCV
4. **Watershed Separation** — Splits touching / overlapping cells — OpenCV
5. **Cell Analysis** — Numbering, area, histogram — scikit-image

OUTPUTS

Results



- Numbered cell contours
- Area of every cell
- Area histogram
- Total cell count
- CSV report

loads weights

SHARED INFRASTRUCTURE · utils/

Device Manager	Error Handling	Logging	Visualisation	Metrics
GPU → CPU auto-fallback	try / except per stage	per-stage timing	heatmaps & overlays	IoU · Dice · PSNR · SSIM

OFFLINE TRAINING · training/

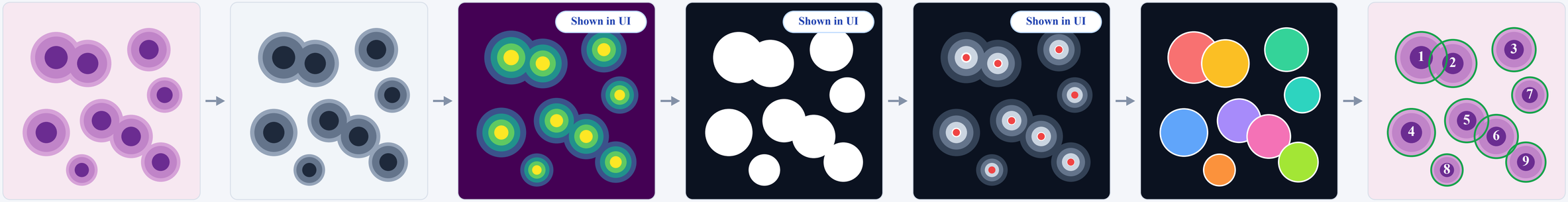
DSB 2018 dataset → **Train U-Net** BCE + Dice loss → **unet_cells.pth** saved weights

02 · DATA FLOW

Image Processing Pipeline

From a stained microscopy image to individually numbered and measured cells — what the image looks like after each stage.

INPUT | PREPROCESS | DEEP LEARNING | CLASSICAL SEGMENTATION | ANALYSIS



0 Input image

Stained microscopy image with touching and overlapping cells.

`PNG / JPG / TIF`

1 Preprocess

Grayscale, CLAHE contrast boost and Gaussian denoising.

`clip limit`

2 U-Net probability

For each pixel: how likely it belongs to a cell (0 – 1).

`pretrained weights`

3 Binary mask

Keep pixels with P > τ, then clean with opening / closing.

`τ, kernel k`

4 Distance map

Distance to background. Peaks (red) mark cell centres = seeds.

`peak ratio r, min dist`

5 Watershed

Regions grow from each seed; borders form where cells touch.

6 Count & measure

Each cell is numbered and its area is measured.

`min area`

Chips show the parameter controlled by a slider at that stage.

Why distance transform + watershed?

1. Touching cells merge into one white blob in the binary mask.
2. Distance transform is highest at each cell’s centre — every cell gets its own peak.
3. Watershed grows a region from every peak; a border is drawn where two regions meet.

Tuning the probability threshold τ

0.05 — 0.95

Too low
cells merge (under-segmented)

Balanced
one region per cell

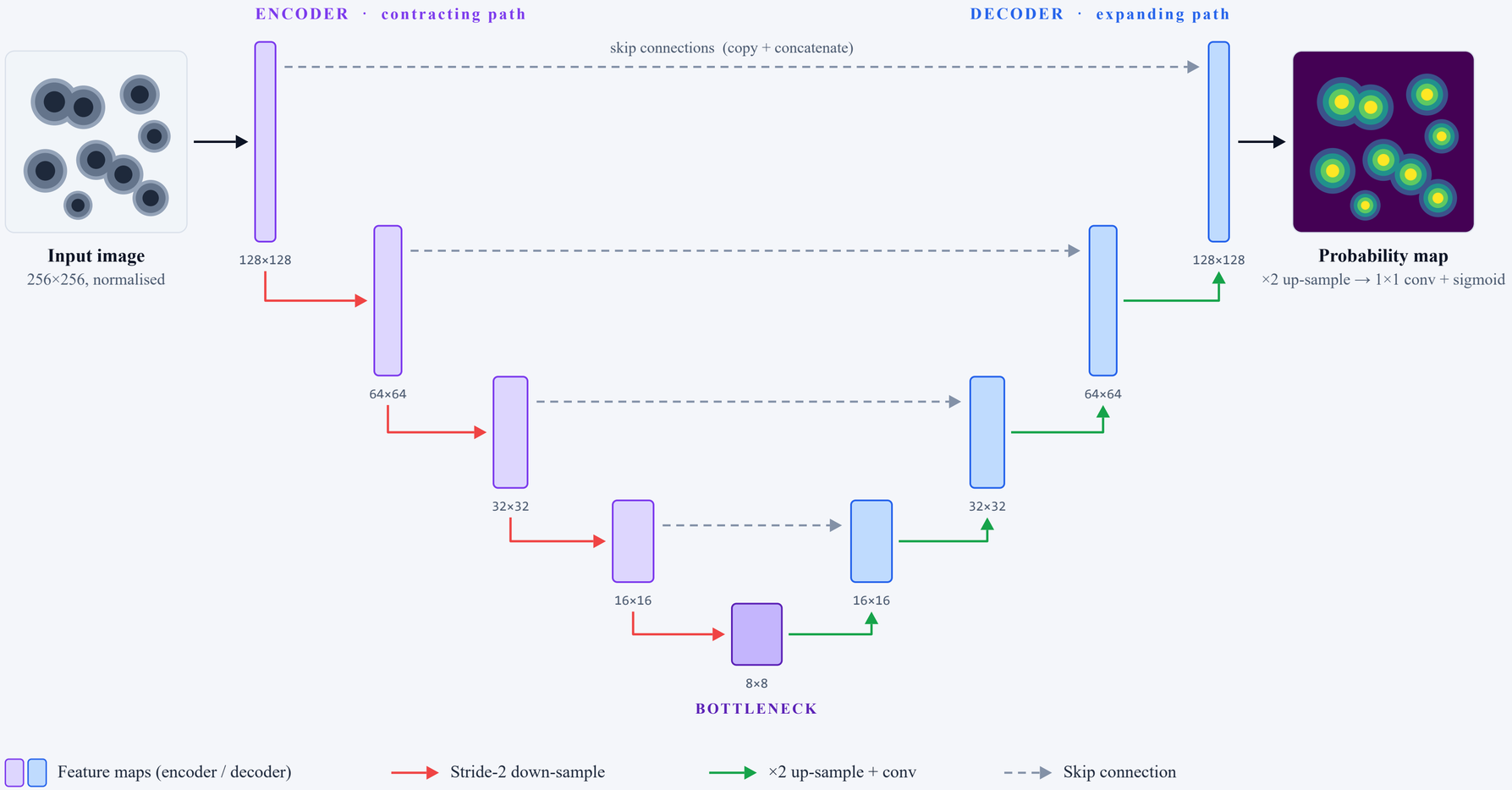
Too high
cells break up or vanish (over-seg.)

Check the heatmap and distance map in the UI before trusting the count.

03 · DEEP LEARNING MODULE

U-Net Segmentation Model & Training

An encoder–decoder network (pretrained ResNet34 encoder) that predicts, for every pixel, the probability of belonging to a cell.



Encoder = ImageNet-pretrained ResNet34 (residual blocks). Each decoder block = ×2 up-sample + two 3×3 conv + BatchNorm + ReLU. Skip connections restore fine cell edges lost by down-sampling.

TRAINING SETUP

Dataset

Kaggle Data Science Bowl 2018 — 670 stained nucleus images with instance masks. Split 80 / 10 / 10.

Preparation

Same Preprocessor as inference (grayscale → 3 ch, CLAHE, blur) · merge instance masks · resize to 256×256 · normalise · augment: flips, rotations, elastic warp, brightness jitter.

Loss function

$$L = BCE + (1 - Dice)$$

$$Dice = 2|P \cap G| / (|P| + |G|)$$

Optimiser

Adam · learning rate 1e-4 · batch 8 · 50 epochs · early stop on validation Dice.

Evaluation

IoU · Dice · precision / recall per cell · cell-count error (%).

Output

Best checkpoint saved to weights/unet_cells.pth and loaded once by UNetSegmenter when the app starts.

Project Structure & Module Interfaces

Folder layout and the public, type-annotated class signatures that each module exposes (Python 3.8+, typing.List / Tuple).

DIRECTORY TREE

```
cell_seg/
├── app.py                  Gradio UI entry point
├── config.py               defaults, paths, @dataclass types
├── requirements.txt
├── core/                   processing modules
│   ├── pipeline.py         orchestrator
│   ├── preprocess.py       CLAHE, blur
│   ├── unet_model.py       U-Net wrapper
│   ├── postprocess.py      threshold + morphology
│   ├── watershed.py        distance transform
│   └── analysis.py         count, area, histogram
├── training/               offline training
│   ├── dataset.py          DSB 2018 loader
│   ├── losses.py           BCE + Dice
│   └── train.py            fine-tuning loop
├── utils/                  shared helpers
│   ├── device.py           GPU / CPU fallback
│   ├── visualize.py        heatmaps, overlays
│   ├── metrics.py          IoU, Dice, PSNR, SSIM
│   └── logger.py           logging, try / except
├── weights/                unet_cells.pth
├── samples/                ≥ 3 test images
├── outputs/                results, CSV
└── tests/                  unit tests
```

class
Preprocessor
`core/preprocess.py`

Normalises colour and contrast so the network sees consistent input.

```
run(img: np.ndarray,
    clip: float = 2.0,
    blur_k: int = 3)
    -> np.ndarray
```

class
UNetSegmenter
`core/unet_model.py`

Loads the pretrained U-Net and predicts a per-pixel probability map.

```
__init__(weights: str,
    device: torch.device)
predict_prob(img: np.ndarray)
    -> np.ndarray   # H×W, 0–1
```

class
MaskRefiner
`core/postprocess.py`

Turns the probability map into a clean binary mask.

```
run(prob: np.ndarray,
    thresh: float, kernel: int,
    min_area: int)
    -> np.ndarray
```

class
WatershedSplitter
`core/watershed.py`

Separates touching cells with distance transform + watershed.

```
run(img: np.ndarray,
    mask: np.ndarray,
    peak_ratio: float, min_dist: int)
    -> Tuple[np.ndarray, np.ndarray]
```

class
CellAnalyzer
`core/analysis.py`

Numbers every cell and measures its area, centroid and bounding box.

```
run(labels: np.ndarray,
    min_area: int)
    -> List[CellInfo]
histogram(cells: List[CellInfo])
    -> Figure
```

class
CellCountingPipeline
`core/pipeline.py`

Runs all stages in order and returns every intermediate result.

```
run(img: np.ndarray,
    params: Params)
    -> PipelineResult
```

SHARED DATA TYPES (@dataclass) & UTILITIES

Params all slider values **CellInfo** id, area, centroid, bbox **PipelineResult** images, cells, count **get_device()** → torch.device

Interactive Gradio Interface

Upload an image, tune the parameters with sliders, and review the intermediate results before trusting the count.

localhost:7860 · Cell Segmentation & Counting

1 Upload image

Drop image here or click to browse

2 Adjust parameters

Probability threshold τ 0.50

Morphology kernel size 5

Peak ratio r 0.40

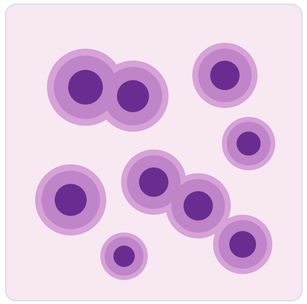
Min peak distance 10 px

Min cell area 50 px²

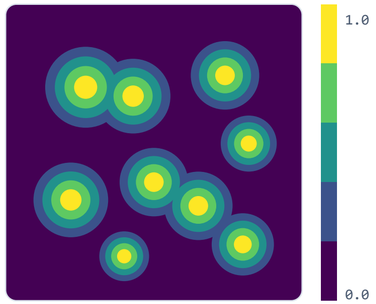
CLAHE clip limit 2.0

Run segmentation | Reset

Original image



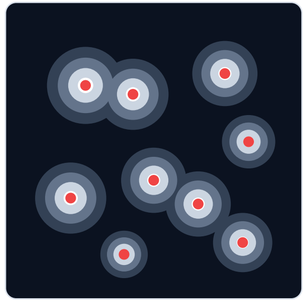
Probability heatmap intermediate



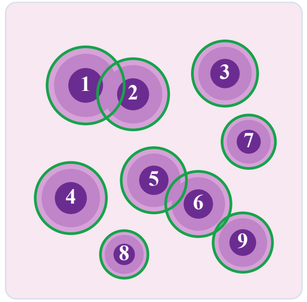
Binary mask intermediate



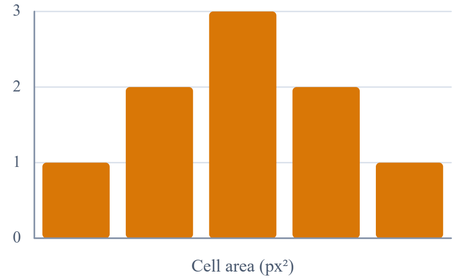
Distance map + seeds intermediate



Segmented & numbered cells



Cell area distribution



Cells detected	Mean area	Median area	Device
9	812 px²	790 px²	CUDA (GPU)

Download CSV

Robustness & Error Handling

Decision flow that keeps the application running when the input, the hardware or the parameters cause problems.

